

LinkedQueue.c

연결 리스트(Linked List)를 기반으로 한 **큐(Queue)** 자료구조의 구현 예제입니다.

개요

이 파일은 `LinkedQueueType` 구조체를 사용하여 동적으로 큐를 구현하고, 기본적인 큐 연산인 `enqueue`, `dequeue`, `is_empty`, `print_queue` 등을 처리합니다.

고정 크기 배열이 아닌 연결 리스트 기반이기 때문에 큐의 크기에 제한이 없습니다.

주요 기능

◇ 큐 초기화

```
void init(LinkedQueueType *q);
```

- `front`와 `rear` 포인터를 `NULL`로 설정하여 빈 큐 생성

◇ 큐 상태 확인

```
int is_empty(LinkedQueueType *q);
```

- `front == NULL`이면 큐가 비어 있음

```
int is_full(LinkedQueueType *q);
```

- 연결 리스트 기반 큐는 항상 공간이 존재하므로 `0` 반환

◇ 큐에 요소 삽입 (enqueue)

```
void enqueue(LinkedQueueType *q, element data);
```

- 새 노드를 동적 생성 후 `rear`에 연결
- 비어 있는 큐일 경우 `front`, `rear`를 모두 해당 노드로 지정

◇ 큐에서 요소 제거 (dequeue)

```
element dequeue(LinkedQueueType *q);
```

- **front** 노드를 제거하고, 다음 노드를 **front**로 설정
- **front**가 **NULL**이 되면 **rear**도 **NULL**로 설정
- 큐가 비어 있으면 오류 메시지 출력 후 프로그램 종료

◇ 큐 출력

```
void print_queue(LinkedQueueType *q);
```

- **front**부터 시작하여 노드 순서대로 출력

🗺 구조체 정의

```
typedef struct QueueNode {
    element data;
    struct QueueNode *link;
} QueueNode;

typedef struct {
    QueueNode *front, *rear;
} LinkedQueueType;
```

- **QueueNode**: 데이터 + 다음 노드를 가리키는 포인터
- **LinkedQueueType**: 전체 큐를 표현하는 구조체

🔗 컴파일 & 실행

```
# 컴파일
gcc -o LinkedQueue LinkedQueue.c

# 실행
./LinkedQueue
```

⚠ 출력 문자열이 한글일 경우 터미널 인코딩 설정이 필요할 수 있습니다.

💡 학습 포인트

- ☒ 연결 리스트 기반 큐 구조 이해
- ☒ 포인터를 활용한 front/rear 관리

- ☒ 동적 메모리 할당/해제의 적절한 사용
- ☒ 예외 상황 처리 (`is_empty`, 오류 메시지 등)

🔍 큐 동작 예시

```
Enqueue(10) → Enqueue(20) → Enqueue(30)
front → [10] → [20] → [30] ← rear
```

```
Dequeue() → returns 10
front → [20] → [30] ← rear
```

⚠️ 개선 포인트

```
q->front = q->rear = 0; // 비추천
```

- 포인터 타입에는 `NULL`을 사용하는 것이 더 안전하고 명확합니다.

```
q->front = q->rear = NULL; // 권장 방식
```
